

参考答案2

13.2

header	↑ 4			
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	24556	Turnamian	Finance	98000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				↑ 6
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.101 The file after insert (24556, Turnamian, Finance, 98000).

header	↑ 2			
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	24556	Turnamian	Finance	98000
record 2				↑ 4
record 3	22222	Einstein	Physics	95000
record 4				↑ 6
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.102 The file after delete record 2.

header	↑ 4			
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	24556	Turnamian	Finance	98000
record 2	34556	Thompson	Music	67000
record 3	22222	Einstein	Physics	95000
record 4				↑ 6
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Figure 13.103 The file after insert (34556, Thompson, Music, 67000).

13.3

The relation *section* with three tuples is as follows:

course_id	sec_id	semester	year	building	room
BIO-301	1	Summer	2010	Painter	514
CS-101	1	Fall	2009	Packard	101
CS-347	1	Fall	2009	Taylor	3128

ID	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-347	1	Fall	2009	A
12345	CS-101	1	Fall	2009	C
17968	BIO-301	1	Summer	2010	null
23856	CS-347	1	Fall	2009	A
45678	CS-101	1	Fall	2009	F
54321	CS-101	1	Fall	2009	A-
54321	CS-347	1	Fall	2009	A
59762	BIO-301	1	Summer	2010	null
76543	CS-101	1	Fall	2009	A
76543	CS-347	1	Fall	2009	A
78546	BIO-301	1	Summer	2010	null
89729	BIO-301	1	Summer	2010	null
98988	BIO-301	1	Summer	2010	null

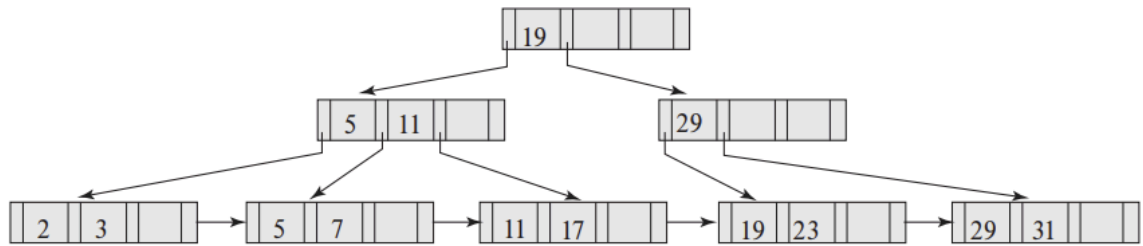
Figure 13.104 The relation *takes* with five students for each section.

BIO-301	1	Summer	2010	Painter	514
17968	BIO-301	1	Summer	2010	null
59762	BIO-301	1	Summer	2010	null
78546	BIO-301	1	Summer	2010	null
89729	BIO-301	1	Summer	2010	null
98988	BIO-301	1	Summer	2010	null
CS-101	1	Fall	2009	Packard	101
00128	CS-101	1	Fall	2009	A
12345	CS-101	1	Fall	2009	C
45678	CS-101	1	Fall	2009	F
54321	CS-101	1	Fall	2009	A-
76543	CS-101	1	Fall	2009	A
CS-347	1	Fall	2009	Taylor	3128
00128	CS-347	1	Fall	2009	A
12345	CS-347	1	Fall	2009	A
23856	CS-347	1	Fall	2009	A
54321	CS-347	1	Fall	2009	A
76543	CS-347	1	Fall	2009	A

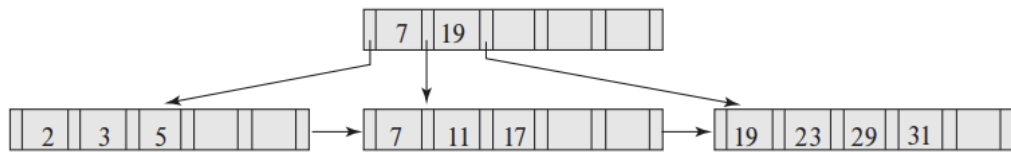
Figure 13.105 The multitable clustering for the above two instances can be taken as:

14.3

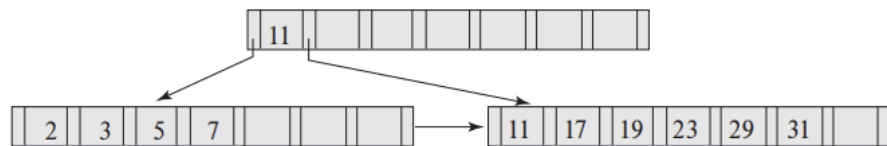
a.



b.



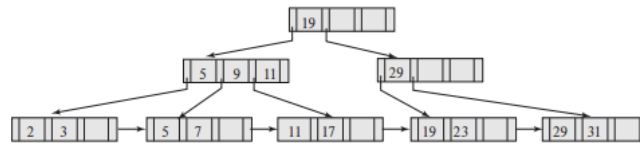
c.



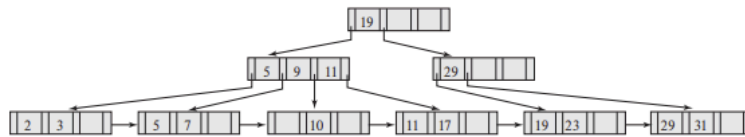
14.4

- With structure Exercise 14.3.a:

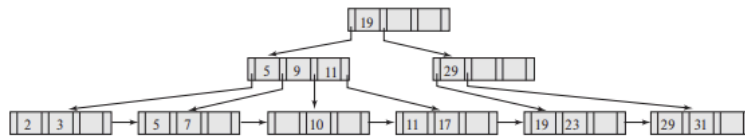
Insert 9:



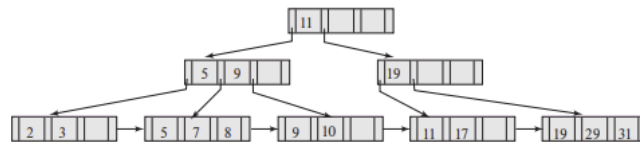
Insert 10:



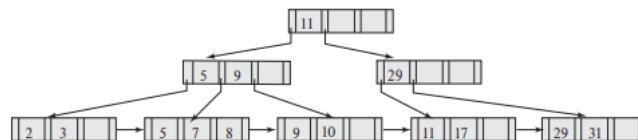
Insert 8:



Delete 23:

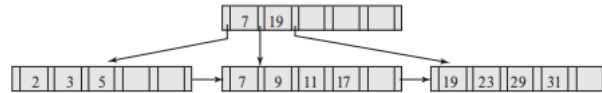


Delete 19:

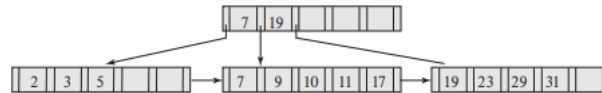


- With structure Exercise 14.3.b:

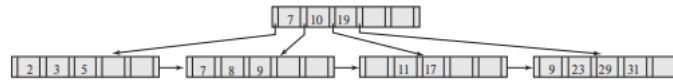
Insert 9:



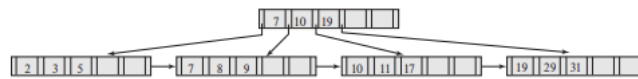
Insert 10:



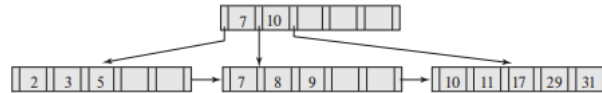
Insert 8:



Delete 23:

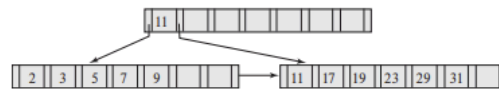


Delete 19:

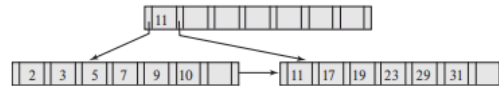


- With structure Exercise 14.3.c:

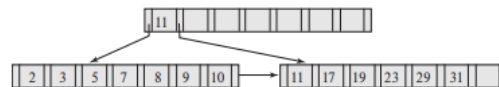
Insert 9:



Insert 10:



Insert 8:



Delete 23:



Delete 19:



14.22

a. When a leaf page is split in a B+-tree file organization, a number of records are moved to a new page. In such cases, all secondary indices that store pointers to the relocated records would have to be updated, even though the values in the records may not have changed.

- b.** Each leaf page may contain a fairly large number of records, and each of them may be in different locations on each secondary index.
- c.** One solution is to store the values of the primary-index search key attributes in secondary indices, in place of pointers to the indexed records. Relocation of records because of leaf-page splits then does not require any update on any secondary index.
- d.** Locating a record using the secondary index now requires two steps: First we use the secondary index to find the primary index search-key values, and then we use the primary index to find the corresponding records. This approach reduces the cost of index update due to file reorganization, although it increases the cost of accessing data using a secondary index.

14.23

Write-optimized indices (such as LSM trees, buffer trees, ...) have very efficient write methods that are much faster than B+-tree's writes. But they offer this write efficiency at the cost of lookup efficiency. Lookup on LSM trees for example costs more than lookup on B+-trees, since a single lookup on LSM trees would require looking up multiple B+-trees.

15.1

We will refer to the tuples (kangaroo, 17) through (baboon, 12) using tuple numbers t_1 through t_{12} . We refer to the j^{th} run used by the i^{th} pass, as r_{ij} . The initial sorted runs have three blocks each. They are:

$$\begin{aligned} r_{11} &= \{t_3, t_1, t_2\} \\ r_{12} &= \{t_6, t_5, t_4\} \\ r_{13} &= \{t_9, t_7, t_8\} \\ r_{14} &= \{t_{12}, t_{11}, t_{10}\} \end{aligned}$$

Each pass merges three runs. Therefore the runs after the end of the first pass are:

$$\begin{aligned} r_{21} &= \{t_3, t_1, t_6, t_9, t_5, t_2, t_7, t_4, t_8\} \\ r_{22} &= \{t_{12}, t_{11}, t_{10}\} \end{aligned}$$

At the end of the second pass, the tuples are completely sorted into one run:

$$r_{31} = \{t_{12}, t_3, t_{11}, t_{10}, t_1, t_6, t_9, t_5, t_2, t_7, t_4, t_8\}$$

15.3

r_1 needs 800 blocks, and r_2 needs 1500 blocks. Let us assume M pages of memory. If $M > 800$, the join can easily be done in $1500 + 800$ disk accesses, using even plain nested-loop join. So we consider only the case where $M \leq 800$ pages.

a. Nested-loop join:

Using r_1 as the outer relation, we need $20000 * 1500 + 800 = 30,000,800$ disk accesses. If r_2 is the outer relation, we need $45000 * 800 + 1500 = 36,001,500$ disk accesses.

b. Block nested-loop join:

If r_1 is the outer relation, we need $\lceil \frac{800}{M-1} \rceil * 1500 + 800$ disk accesses. If r_2 is the outer relation, we need $\lceil \frac{1500}{M-1} \rceil * 800 + 1500$ disk accesses.

c. Merge join:

Assuming that r_1 and r_2 are not initially sorted on the join key, the total sorting cost inclusive of the output is $B_s = 1500(2\lceil \log_{M-1}(1500/M) \rceil + 2) + 800(2\lceil \log_{M-1}(800/M) \rceil + 2)$ disk accesses. Assuming all tuples with the same value for the join attributes fit in memory, the total cost is $B_s + 1500 + 800$ disk accesses.

d. Hash join:

We assume no overflow occurs. Since r_1 is smaller, we use it as the build relation and r_2 as the probe relation. If $M > 800/M$, i.e., no need for recursive partitioning, then the cost is $3(1500 + 800) = 6900$ disk accesses, else the cost is $2(1500 + 800)\lceil \log_{M-1}(800) - 1 \rceil + 1500 + 800$ disk accesses.

16.5

- The relation resulting from the join of r_1 , r_2 , and r_3 will be the same no matter which way we join them, due to the associative and commutative properties of joins. So we will consider the size based on the strategy of $((r_1 \bowtie r_2) \bowtie r_3)$. Joining r_1 with r_2 will yield a relation of at most 1000 tuples, since C is a key for r_2 . Likewise, joining that result with r_3 will yield a relation of at most 1000 tuples because E is a key for r_3 . Therefore, the final relation will have at most 1000 tuples.
- An efficient strategy for computing this join would be to create an index on attribute C for relation r_2 and on E for r_3 . Then for each tuple in r_1 , we do the following:
 - a. Use the index for r_2 to look up at most one tuple which matches the C value of r_1 .
 - b. Use the created index on E to look up in r_3 at most one tuple which matches the unique value for E in r_2 .

16.6

The estimated size of the relation can be determined by calculating the average number of tuples which would be joined with each tuple of the second relation. In this case, for each tuple in r_1 , $1500/V(C, r_2) = 15/11$ tuples (on the average) of r_2 would join with it. The intermediate relation would have $15000/11$ tuples. This relation is joined with r_3 to yield a result of approximately 10,227 tuples ($15000/11 \times 750/100 = 10227$). A good strategy should join r_1 and r_2 first, since the intermediate relation is about the same size as r_1 or r_2 . Then r_3 is joined to this result.

17.6

There is a serializable schedule corresponding to the precedence graph since the graph is acyclic. A possible schedule is obtained by doing a topological sort, that is, T_1, T_2, T_3, T_4, T_5 .

17.15

a.

There are two possible executions: $T_{13} T_{14}$ and $T_{14} T_{13}$.

Case 1:

	A	B
initially	0	0
after T_{13}	0	1
after T_{14}	0	1

Consistency met: $A = 0 \vee B = 0 \equiv T \vee F = T$

Case 2:

	A	B
initially	0	0
after T_{14}	1	0
after T_{13}	1	0

Consistency met: $A = 0 \vee B = 0 \equiv F \vee T = T$

b.

Any interleaving of T_{13} and T_{14} results in a non-serializable schedule.

T_1	T_2
read (A)	read (B)
	read (A)
read (B)	
if $A = 0$ then $B = B + 1$	if $B = 0$ then $A = A + 1$
write (B)	write (A)
T_{13}	T_{14}
read (A)	read (B)
	read (A)
read (B)	
if $A = 0$ then $B = B + 1$	if $B = 0$ then $A = A + 1$
write (B)	write (A)

c. There is no parallel execution resulting in a serializable schedule. From part a. we know that a serializable schedule results in $A = 0 \vee B = 0$. Suppose we start with T_{13} **read**(A). Then when the schedule ends, no matter when we run the steps of T_2 , $B = 1$. Now suppose we start executing T_{14} prior to completion of T_{13} . Then T_2 **read**(B) will give B a value of 0. So when T_2 completes, $A = 1$. Thus $B = 1 \wedge A = 1 \rightarrow \neg(A = 0 \vee B = 0)$. Similarly for starting with T_{14} **read**(B).

18.2

a. Lock and unlock instructions:

```

T34:      lock-S(A)
           read(A)
           lock-X(B)
           read(B)
           if A = 0
           then B := B + 1
           write(B)
           unlock(A)
           unlock(B)

T35:      lock-S(B)
           read(B)
           lock-X(A)
           read(A)
           if B = 0
           then A := A + 1
           write(A)
           unlock(B)
           unlock(A)

```

b. Execution of these transactions can result in deadlock. For example, consider the following partial schedule:

T_{34}	T_{32}
lock-S (A)	
	lock-S (B)
	read(B)
read(A)	
lock-X (B)	
	lock-X (A)

The transactions are now deadlocked.